

DataTable is 25 years old. Nobody ever reimagined it.

It shipped with .NET Framework 1.0 in 2002 and still sits between an enormous number of business applications and their databases. It has been maintained for a quarter of a century. It has never been rebuilt.

Benchmarks: .NET 10 x64, workstation GC Reproducible: `dotnet run -c Release -- article`



70% LESS MEMORY 1M ROWS, 10 COLUMNS	2.9× FASTER LOAD LIKE-FOR-LIKE, 66 COLS
5× LESS GARBAGE SENDING A TVP	18.6× FASTER DELETE 10% OF 200K ROWS

Every figure below is measured, not modelled. Sources and caveats are stated inline.

The awkward part is where it sits. `System.Data.DataTable` is rarely at the edge of a system — it is in the middle: the currency you load a query into, pass around, cache, and hand back to SQL Server as a table-valued parameter. If a table sits in memory for the life of a request, or worse for the life of a cache entry, its shape decides your memory ceiling, your GC pressure, and a surprising slice of your latency.

A data structure in that position is worth rebuilding from the ground up rather than tuning around. So I rebuilt it: column-oriented, strongly typed, allocation-lean. Here is what fell out.

First, the thing that makes the old design expensive

`DataTable` is **row-oriented**. Each row is a heap object holding an `object[]` of its cells. Every value-typed cell — every `int`, `bool`, `DateTime`, `decimal` — gets **boxed**: wrapped in a heap object so it can live in an `object[]`.

A box is not free, and the price is paid per cell. On x64, boxing an `int` costs 24 bytes — an 8-byte object header, an 8-byte method-table pointer, and 4 bytes of payload rounded up to 8. Add the 8-byte array slot pointing at it and one 4-byte integer occupies 32 bytes.

1,000,000 INT32 CELLS WRITTEN, THEN READ BACK

STORAGE	MEMORY	TIME	BYTES / CELL
Boxed <code>object[]</code>	30.5 MB	11,539 μ s	32.0
Typed <code>Int32[]</code>	3.8 MB	462 μ s	4.0
Difference	8 $\times$ memory	25 $\times$ time	—

Same million integers. Eight times the memory, twenty-five times the time.

Now multiply by *cells*, not rows. A 1,000,000-row table with 10 value-typed columns performs **ten million** boxing operations to fill and ten million unboxing operations to read. That is not a micro-optimisation you are leaving on the table. That *is* the table.

The idea: turn the table ninety degrees

Instead of storing a table as rows of `object`, store it as **columns of T**.

A column of a million `int` s becomes an `Int32[]`, chunked to stay off the large object heap. A column of a million `bool` s becomes a bitmap. Nothing is boxed, because nothing ever needs to be an `object`. And a row stops being an object at all — it becomes a lightweight view synthesized on demand from a table reference and an index.

Six things follow from that, and each is worth stating on its own.

01 Column-oriented, strongly typed storage

Each column owns a typed array of exactly the right width. An `Int16` column costs 2 bytes per row, not 24 plus a pointer. There is no per-row object and no `object[]` per row, so the fixed overhead a row-oriented table pays *before storing any data* simply is not there.

02 A Boolean costs one bit

Booleans are bit-packed, 64 rows to a `UInt64`. A million-row `bool` column is 125 KB of bits rather than a million boxed objects. Null tracking uses the same trick — one bit per row, set only when the cell *is* null, so a nullable column that never actually receives a null never allocates a single word for it.

03 Strongly typed methods, so nothing boxes on the hot path

Resolve the column handle once, outside your loop, then read and write through it:

```

DataColumn<int> quantity = table.Columns.GetColumn<int>("Quantity");
long total = 0;
for (int i = 0; i < table.Rows.Count; i++)
{
    total += quantity.Get(i);    // array index. no cast, no box, no type check.
}

```

That is an array index and a bit test. The `object` indexer is still there for compatibility — it just is not where you put a loop over a million rows.

04 Deletion is a tombstone, not a shift

`System.Data` removes a row by physically removing it from an index, which means shifting everything after it. Delete a tenth of a large table and you pay for that repeatedly. Here a deleted row is **tombstoned**: a bit flips, the visible row count drops, and a Fenwick tree translates logical positions to physical ones in $O(\log n)$. Nothing moves, nothing is copied, and every row handle you were already holding stays valid.

DELETE EVERY 10TH ROW OF 200,000, THEN READ THE SURVIVORS

IMPLEMENTATION	TIME
<code>System.Data.DataTable</code>	165.3 ms
<code>ColumnStore.Data</code>	8.9 ms
	18.6× faster

05 Loading is faster, because the loader is typed too

Loading from an `IDataReader` is where most tables get filled, so the type decision happens **once per column at schema time**, not once per cell. The loader builds one pre-bound delegate per field that has already captured the ordinal, the cast column and the typed getter, so the per-row loop does nothing but invoke them. `reader.GetInt32(3)` goes straight into an `Int32[]`. Nothing boxes.

06 Table-valued parameters get faster for the same reason

Sending rows to SQL Server via a TVP means producing a stream of `SqlDataRecord`s. From a `DataTable`, every cell you read is already boxed and stays boxed on the way into the record. From a typed column, the value goes from an array straight into `SetInt32`.

PRODUCING 200,000 SQLDATARECORDS

SOURCE	TIME	MS / ROW	ALLOCATED	BYTES / ROW
<code>System.Data.DataTable</code>	35,012 μ s	0.175	40.4 MB	212.0
<code>ColumnStore.Data</code>	19,851 μ s	0.099	7.6 MB	40.0
	1.8× faster	—	5× less	—

The allocation figure is the one to notice: 40 bytes per row instead of 212 is GC pressure your whole process stops paying, not just this loop.

The numbers, across row counts and column types

A mixed 10-column table — `int`, `string`, `bool`, `decimal`, `DateTime`, `long`, `Guid`, `short`, `double`, `bool` — populated identically both ways.

ROWS	MEMORY USED, AS A SHARE OF SYSTEM.DATA	SAVED
1,000		0.2 → 0.1 MB 70%
10,000		2.8 → 0.7 MB 74%
100,000		25.2 → 7.0 MB 72%
1,000,000		233.5 → 69.2 MB 70%

234 MB down to 69 MB. The saving holds at around 70% from a thousand rows to a million — it is structural, not a trick that only pays at scale.

BUILD TIME, SAME TABLE

ROWS	SYSTEM.DATA	COLUMNSTORE	FASTER
1,000	0.3 ms	0.0 ms	9.6×
10,000	4.9 ms	0.3 ms	15.1×
100,000	53.6 ms	3.4 ms	15.6×
1,000,000	1,001.7 ms	46.4 ms	21.6×

Two honest notes. The ColumnStore arm uses the typed `Set` path and the System.Data arm uses `Rows.Add(object[])`, because that is how each is idiomatically populated — so 21.6× is best-idiomatic against best-idiomatic, not the same code twice. When both are loaded from the same `IDataReader` through their respective loaders, the like-for-like figure is a more sober 2.9× faster and 51% less memory on a 66-column production schema at 50,000 rows.

ONE COLUMN OF 1,000,000 ROWS, BY TYPE — RETAINED BYTES PER ROW

TYPE	SYSTEM.DATA	COLUMNSTORE	RATIO
Boolean	145.8	0.4	370×
Byte	145.8	1.5	96×
Int16	146.9	2.5	58×
Int32	149.0	4.5	33×
Int64 / Double / DateTime	153.2	8.5	18×
Decimal	161.6	16.5	10×
Guid	185.0	16.5	11×

Read this one carefully: it is a *single-column* table, so the `System.Data` side is dominated by the fixed per-row cost – the row object and its `object[]` – which is exactly the overhead that disappears. That is why every type looks alike on the left and tracks its true width on the right. `Boolean` at 0.4 bytes is the one bit plus chunk bookkeeping.

What you actually have to change

The API is shaped to match `System.Data . Columns.Add("Id", typeof(int)) , Rows.Add(1, "widget") , row["Name"] , foreach (DataRow row in table.Rows) , Clone , Copy , ImportRow , Select() , row.Delete()` – all present, all behaving the same way, down to details that ported code silently depends on: a null cell reads back as `DBNull.Value` , `Columns["missing"]` returns `null` , `Columns[99]` throws `IndexOutOfRangeException` .

Change the `using` . That is the port. Every shape `System.Data` code assigns a cell in compiles and writes:

```
table.Rows[0]["Name"] = "changed";
table.Rows[0].ItemArray = values;
table.Rows[0]["A"] = table.Rows[1]["A"];
foreach (DataRow row in table.Rows) { row["Name"] = "changed"; }
DataRow row = table.Rows[0]; row["Name"] = "changed";
table.Select()[0]["Name"] = "changed";
```

That took one deliberate piece of design, and one measurement that overruled the obvious implementation.

Why a row indexer has to return a reference

`DataRow` is a readonly struct – that is where the memory saving lives – and C# only allows assignment through a property whose receiver is a *variable*. A by-value struct return is not one: the write would land in a temporary and be discarded, so the compiler refuses it. So `Rows[i]` returns `ref DataRow` , and the receiver becomes a variable again.

A reference needs somewhere real to point, and the cheap choice – one scratch slot reused by every call – is **wrong in a way nothing reports**. In `rows[0]["Name"] = rows[1]["Name"]` the compiler takes the target's address first, then evaluates the right-hand side, which overwrites that same slot with row 1. The write lands in *row 1*; row 0 is left untouched and nothing throws. I reproduced that before choosing the design, and there is now a test whose only job is to fail if anyone optimises back to it.

So each row gets its own view slot, chunked at 1,024 rows – small enough to stay off the large object heap, and never reallocated, so a reference taken earlier stays valid however many rows are appended after it.

The store is built lazily on the first `Rows[i]` call and released by `Clear`, and the library's own internals never touch it. So the cost is real, bounded, and paid only by tables that actually index a row:

1,000,000 ROWS × 10 COLUMNS – WHAT THE ROW-VIEW STORE COSTS

ACCESS PATTERN	RETAINED	VS SYSTEM.DATA
foreach and typed handles only	69.2 MB	70.4% saved
Row indexer used on every row	84.5 MB	63.8% saved
Cost of the views	16.0 bytes/row	–

Write your hot loops against a typed column handle and you pay nothing for compatibility you are not using. Write them the way your existing code already does, and you still keep two thirds of the saving.

And what is deliberately out of scope

Being honest about this is the point, not a disclaimer. There is no expression engine, no constraint engine, no relations, no change tracking, no XML. So `Select("Id > 5")`, `Compute`, `DefaultView`, `PrimaryKey`, `Constraints`, `Relations`, `Merge`, `GetChanges`, `RejectChanges` and `ReadXml` all throw `NotSupportedException` – **with a message naming the alternative.**

They throw rather than quietly doing nothing on purpose. A `Select(filter)` that returned every row, or a `Merge` that appended instead of matching, would hand back *wrong answers*; throwing hands back a stack trace pointing at the line to change. Table events refuse at subscription time for the same reason: firing them would cost per row, and never firing them would leave your handler silently dead.

A few members are no-ops, and that is exact rather than lazy – `AcceptChanges` has nothing to commit when writes go straight into column storage, and `BeginLoadData` has no indexes to suspend.

The takeaway

A quarter of a century is a long time for a data structure to sit in the middle of everything without anyone asking whether its shape is still the right one. Row-oriented storage with boxed cells made sense in 2002, when the alternative was worse. It does not make sense now.

Turn it ninety degrees and the numbers move by multiples, not percentages: **~70% less memory**, **2.9× faster to load** like-for-like, **1.8× faster** to send as a TVP with **5× less garbage**, **18.6× faster** to delete from. Change a `using` and all of that arrives without touching your logic. Resolve a typed column handle in your hot loops and you keep the last of it too.

Every figure in this article comes from benchmarks in the repository and can be reproduced with `dotnet run -c Release -- article`.

Measured on .NET 10, workstation GC, x64. Memory is retained bytes after a forced gen-2 collection, not working set.